

GameCraft-Bench: 智能体能否在真实游戏引擎中端到端地构建可玩游戏?

Tongxu Luo^{1,2*}, Rongsheng Wang^{1*}, Jiayi Bi^{2,4*†}, Chenming Xu^{1,2*}, Zhengyang Tang^{1,3*}, Jianlong Chen¹, Juhao Liang¹, Ke Ji¹, Shuqi Guo^{1,2}, Yuhao Du^{1,2}, Fan Bu^{1,2}, Wenyu Du⁵, Xiaotong Zhang⁴, Kyle Li⁷, Shaobo Wang⁶, Linfeng Zhang⁶, Yuxuan Liu³, Xin Lai³, Chenxin Li³, Yiduo Guo³, Zhexin Zhang³, Xinyuan Wang³, Tianyi Bai³, Ziniu Li³, Benyou Wang^{1,2‡}

¹The Chinese University of Hong Kong, Shenzhen ²Shenzhen Loop Area Institute

³Hunyuan Team, Tencent ⁴USTB ⁵DualverseAI ⁶SJTU ⁷NUS

* 共同贡献 † 实习于 SLAI 期间完成 ‡ 通讯作者



图 1 GameCraft-Bench 中由编码智能体生成的可玩游戏，覆盖 15 个游戏族、140 个任务。

摘要

游戏生成是编码智能体的一项新兴应用，要求模型将自然语言规格转化为可玩的交互系统。与传统编码任务不同，游戏生成发生在游戏引擎之中，脚本、场景、资源、渲染与运行时交互必须共同产出连贯的游戏体验。我们将端到端游戏生成形式化为这样一个问题：在目标环境中产出一个完整的游戏产物，并通过可观察的玩家—游戏交互来实现给定规格。我们认为，评测这一场景需要满足三项准则：**引擎落地**（Engine Grounding）、**产物完整性**（Artifact Completeness）与**交互式验证**（Interactive Verification）。我们提出一套以交互为基础的评测框架，通过重放演示与量规引导的多模态评判来评估可执行的游戏过程。我们将该框架实例化为 **GameCraft-Bench**，这是一个涵盖 15 个游戏族、140 个 Godot 任务的基准。对前沿编码智能体的评测表明，端到端游戏生成仍极具挑战：最强的智能体也仅取得 41.46%，且大多数智能体得分低于 40%。进一步分析表明，尽管智能体常能实现可识别的游戏机制，但它们仍难以交付内容充分、视觉反馈可用、呈现连贯的完整游戏。演示、代码与数据见 <https://tongxuluo.github.io/gamecraft-bench-website>。

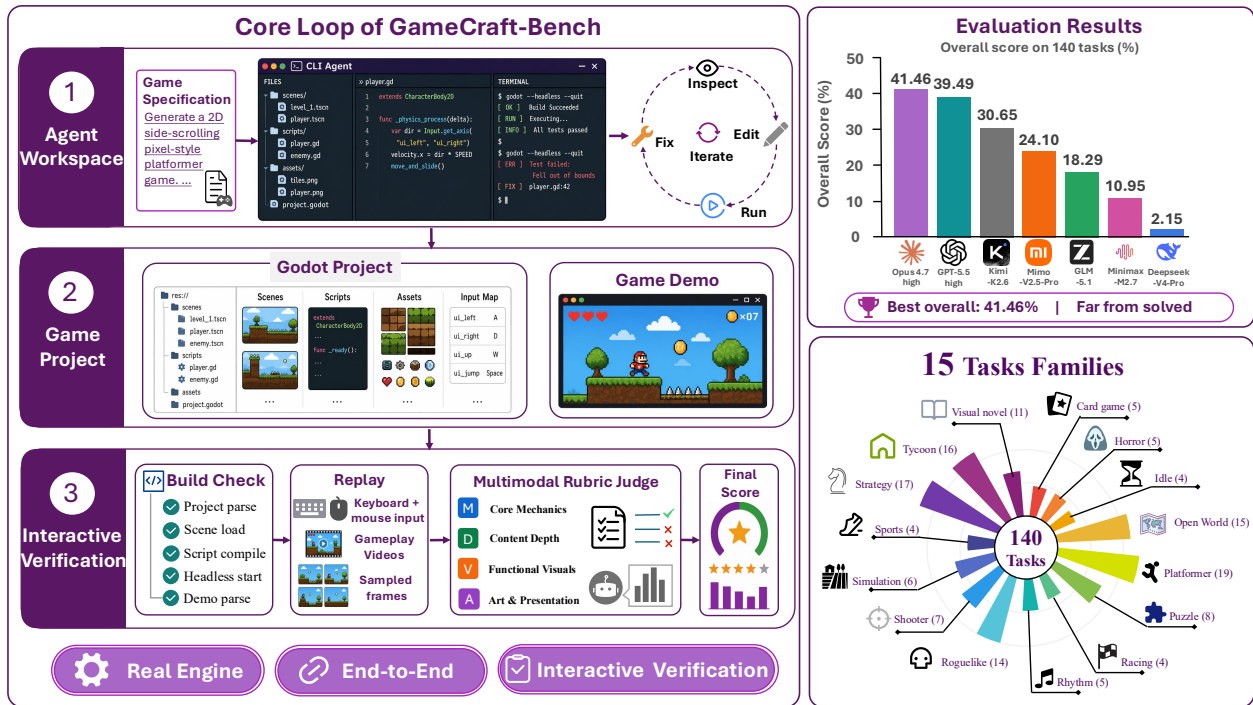


图 2 GameCraft-Bench 总览。智能体将自然语言游戏规格转化为完整的 Godot 项目，验证器重放提交的交互轨迹，量规引导的评判对游戏过程证据进行评分。

1 引言

游戏生成 [1–4] 将创意意图转化为交互式系统：玩家不仅观看结果，还会在其中行动，并期待这个世界做出回应。近期的编码智能体 [5–8] 使自动化游戏生成日益变得可行，因为它们能够检查文件、编辑项目、运行工具，并依据执行反馈进行迭代。然而，评估生成的游戏与评估普通代码并不相同：成败取决于一个产物能否被构建、启动、游玩，并作为一个交互式系统被观察。

游戏生成的准则。 端到端的游戏生成必须回答三个递进的问题：智能体在何处进行开发，它交付什么产物，以及最终的游戏如何被评估。这些问题引出三条准则：**准则 I：引擎落地**，它要求在一个真实的游戏引擎中，借助其原生文档和资源进行开发；**准则 II：产物完整性**，它要求智能体连同所有必要产物一并生成一个可启动的游戏；以及**准则 III：交互式验证 (Interactive Verification)**，它要求通过运用玩家输入与玩法重放进行直接交互来开展评估。这些准则合在一起，将评估从**代码正确性**转向**交互式系统正确性**，为引擎落地、完整构建且基于行为评估的游戏生成确立了一份基准契约。

现有基准未能满足这些准则。 现有的游戏生成基准各自满足这份契约的一部分，但都未能同时满足全部三条准则。OpenGame-Bench [2] 通过要求智能体根据开放式提示生成完整游戏来满足产物完整性；然而，它面向的是网页游戏而非真实引擎项目，且不通过玩法交互来评判游戏。GameDevBench [9] 通过将评估迁移到真实引擎中来满足引擎落地；然而，它研究的是源于教程的局部修改和确定性测试，而非完整的游戏构建与基于交互的可玩性。与我们的设定最为接近的 WebGameBench [10] 通过浏览器交互来评估 Web 游戏，但它仍处于真实游戏引擎开发之外，并依赖评估侧的探索。因此，现有基准未能涵盖表 1 所刻画的完整设定：智能体能否将用户意图一路贯彻到一个完整的、引擎原生的游戏产物，且其行为通过交互来评判。

基准	准则 I 引擎落地	准则 II 产物完整性	准则 III 交互式验证
GameDevBench [9]	Godot	× (单步操作)	×
OpenGame-Bench [2]	Web	✓	×
WebGameBench* [10]	Web	✓	✓
GameCraft-Bench	Godot	✓	✓

表 1 沿三条准则与现有游戏生成基准的比较。* 值得注意的是，WebGameBench [10] 与我们的工作是并行进行的：它评估交付的 Web 游戏，而 **GameCraft-Bench** 面向的是专用游戏引擎中的完整项目。

GameCraft-Bench 如何填补这一空白。 我们提出 **GameCraft-Bench**，这是一个旨在同时满足游戏生成三条准则的端到端基准。它将开发落地于一个真实游戏引擎（Godot），评估完整的可启动游戏项目而非局部产物，并通过重放交互而非仅靠静态检查来验证生成的游戏。该基准包含跨越 15 个游戏族系的 140 个任务，涵盖从机制和进程系统到视觉呈现的多样化玩法需求。通过统一**引擎落地、产物完整性与交互式验证**，**GameCraft-Bench** 评估编码智能体能否将自然语言的游戏规格转化为可玩的游戏体验。

对 GameCraft-Bench 的观察。 我们在 **GameCraft-Bench** 上评估了前沿编码智能体，发现端到端游戏生成远未得到解决。即便是最强的智能体，总体得分也仅为 41.46%，而大多数受测智能体的得分要低得多。这些低分并非由某一项缺失的能力所致：智能体在核心机制、内容深度、功能性视觉以及美术与呈现等方面均有失败，且其表现在不同游戏族系之间差异显著。这些结果表明，当前的编码智能体往往能够生成类游戏软件的片段，但仍难以可靠地遵循一份游戏规格，将其一路落实为一个真实引擎中可玩且视觉连贯的产物。

贡献。 我们将贡献总结如下：

- 我们将端到端游戏生成形式化为将游戏规格转化为可玩交互式系统的问题，并为其评估确定了三条准则：**引擎落地、产物完整性与交互式验证**。
- 我们提出了一个以交互为基础的评估框架，它通过可执行的玩法证据来验证生成的游戏，将基于重放的交互协议与对机制、内容、视觉功能和呈现质量的量规引导式评估相结合。
- 我们将该框架实例化为 **GameCraft-Bench**，这是一个在 Godot 中包含跨越 15 个游戏族系的 140 个任务的基准，其中智能体必须根据自然语言的游戏规格生成完整的游戏项目。
- 我们对前沿编码智能体进行了基准测试，并提供诊断性分析，表明当前系统在可靠的端到端游戏生成方面仍相去甚远，其局限性涵盖机制、内容深度、视觉反馈、呈现质量以及任务完成行为。

2 怎样才算一个好的游戏生成基准？

2.1 问题定义

AI 游戏生成研究的是智能体如何把高层次的设计意图转化为一个可玩的交互系统。如图 3 所示，智能体接收一份游戏规格，在开发与运行时环境中进行操作，并产出一个可玩的游戏产物。我们将这一过程抽象为

$$x = (s, \mathcal{E}) \mapsto y = G,$$

其中 s 是一份描述目标玩家体验的游戏规格，包括规则、机制、目标、内容与呈现； $\mathcal{E}$ 是目标开发与运行时环境； G 是生成出的游戏产物。一个成功的输出 G 应当能够在 $\mathcal{E}$ 中启动，并应通过对玩家动作的可观察响应来实现 s 。

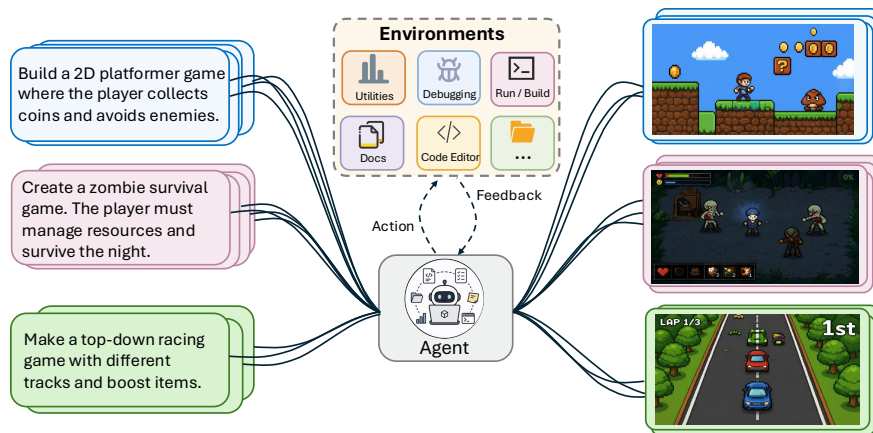


图 3 AI 游戏生成的问题定义。智能体通过在开发与运行时环境 $\mathcal{E}$ 中行动，将自然语言游戏规格 s 转化为可玩的游戏产物 G 。

2.2 三条准则。

一个好的基准必须保留映射 $(s, \mathcal{E}) \mapsto G$ 的完整含义。这要求对评测问题的三个空间加以约束：**开发空间**，即智能体在何处构建游戏；**输出空间**，即智能体必须交付何种产物；以及**评测空间**，即交付的产物如何被评判。对于游戏生成而言，这些空间引出三条准则：**引擎落地**、**产物完整性与交互式验证**。如图 4 所示，这些准则在整体上是完备的，因为它们覆盖了游戏生成的完整成功路径：在环境内部进行生成、交付一个游戏产物，以及通过交互进行验证。如果其中任何一个空间未被约束，基准就不再评测端到端的游戏生成：它可能退化为玩具式的开发设定、不完整的产物，或者错过游玩失败的静态评估。三条准则合在一起，把评测从代码正确性或视觉合理性转向交互系统正确性。

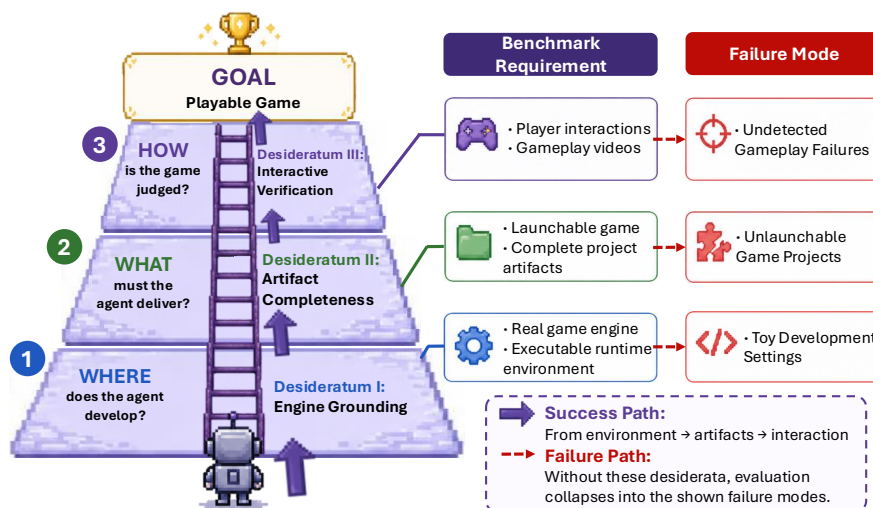


图 4 评测端到端游戏生成的三条准则。

准则 I 引擎落地。 游戏应当在一个具体的游戏生成与运行时环境内被评测。

游戏行为并非仅由源代码定义，而是由引擎层面的语义所定义，例如场景层级、脚本生命周期、资源加载、输入分发、物理、渲染、项目配置以及启动流程。因此，引擎落地要求基准保留这些运行时约束，从而使智能体必须解决真实游戏生成中的集成问题，而不是一项只模仿游戏逻辑的抽象编程

练习。若没有引擎落地，评测可能退化为玩具式的程序合成：可能产出局部上看似合理的代码，却没有证明它能在游戏实际运行的操作环境内工作。

准则 II 产物完整性。 游戏应当作为一个完整的可启动产物被评测。

只有当所需的各个组件被组装成一个可运转的整体时，可玩性才会涌现：项目元数据、入口场景、脚本、资源、UI 元素、输入映射、配置文件以及运行时资源，必须全部齐备并相互连接。因此，产物完整性把可启动的游戏项目作为评测单元，而不是孤立的脚本、精灵图、机制原型，或那些仍需人工进一步组装的视觉上看似合理的场景。这并不要求一个打磨完善的商业游戏，但产物必须足够自包含，以使目标游戏循环能够在目标环境中运行。若没有这条准则，基准可能会过度肯定那些孤立来看像游戏、却在系统边界处失败的局部工作——而正是在系统边界处，一款游戏才必须成为一种可运行的体验。

准则 III 交互式验证。 游戏最终应当依据它们被交互式游玩时所发生的情况来评判。

游戏的决定性特征是动作—响应循环：玩家行动，系统更新，世界呈现出维系目标、反馈、挑战、进程或叙事的后果。许多重要的失败在这一循环被实际运行之前是不可见的，包括无响应的控制、错误的碰撞、不活动的敌人、无法到达的目标、缺失的 UI 反馈、损坏的状态转换，或在游玩过程中变得难以辨别的视觉场景。因此，交互式验证要求基准评判被观察到的游玩行为，而不仅仅依赖静态检查：产物必须被运行、被施以动作，并作为一个交互系统被观察，无论是通过人工游玩、脚本化输入、回放的演示、习得的策略，还是另一种标准化的交互流程。若没有这条准则，评测可能会认证那些在游玩前看起来正确、却恰恰在游戏必须响应玩家时失败的产物。

3 GameCraft-Bench

3.1 任务定义

GameCraft-Bench 在真实游戏引擎的设置中实例化了通用的游戏生成映射 $(s, \mathcal{E}) \mapsto G$ 。每个任务被定义为

$$\tau = (s, \mathcal{E}, \rho),$$

其中 s 是提供给智能体的游戏规格， $\mathcal{E}$ 是目标开发与运行时环境，而 ρ 是验证器所使用的隐藏评分量规。在 **GameCraft-Bench** 中，环境 $\mathcal{E}$ 被具体化为

$$\mathcal{E} = (\mathcal{R}, \mathcal{W}, \mathcal{A}, \mathcal{C}),$$

其中 $\mathcal{R}$ 是 Godot 引擎运行时与工具链， $\mathcal{W}$ 是可编辑工作区， $\mathcal{A}$ 是共享资源接口，而 $\mathcal{C}$ 是提交契约。智能体观察到 $(s, \mathcal{E})$ ，但观察不到 ρ 。

给定 $(s, \mathcal{E})$ ，智能体必须生成

$$y = (G, \Pi),$$

其中 G 是一个完整的 Godot 项目，而 $\Pi = \{\pi_i\}_{i=1}^n$ 是一组可重放的交互轨迹。值得注意的是，这些轨迹本身并不属于游戏产物；它们为评测提供标准化的交互证据。验证器在 $\mathcal{R}$ 中启动 G 并重放 Π ，以获得游戏过程观测：

$$O = \text{Replay}_{\mathcal{R}}(G, \Pi).$$

一次成功的提交，是指其被观测到的行为 O 在 $\mathcal{E}$ 的约束下实现了规格 s 。

3.2 实现

GameCraft-Bench 将每个任务实现为图 5 所示的五阶段流水线。该流水线遵循一个评测实例的完整生命周期：打包一个任务，由智能体构建并提交一个游戏，验证器检查该提交是否可执行，将重放

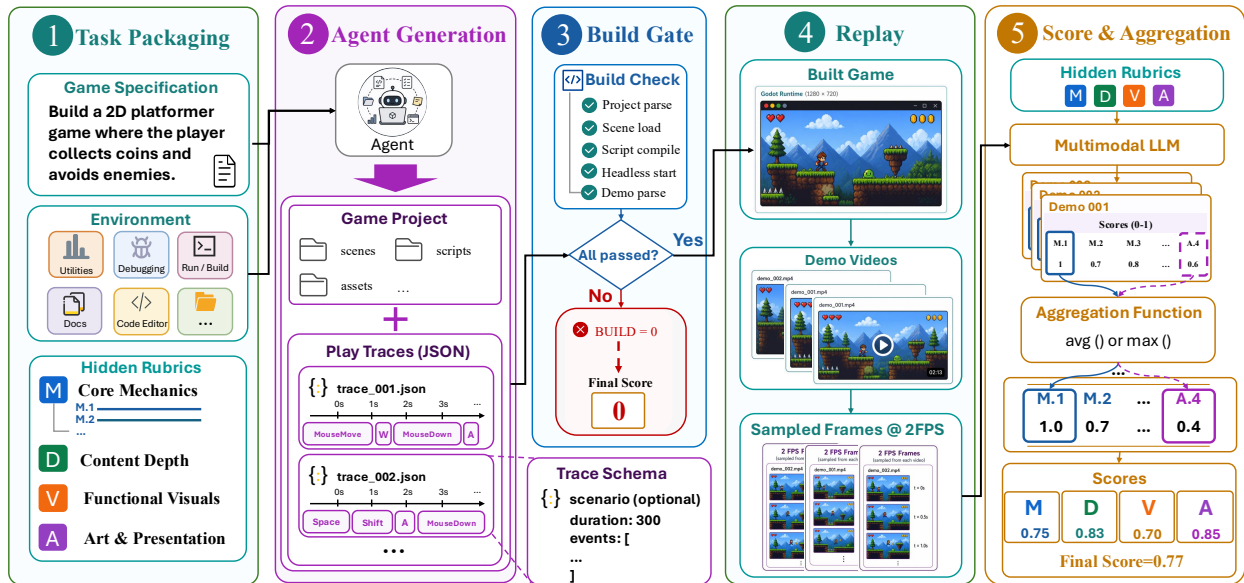


图 5 GameCraft-Bench 的端到端评测流水线。一个任务打包了游戏规格、Godot 环境与隐藏评分量规；智能体生成一个完整的游戏项目与重放轨迹；验证器检查可启动性，将轨迹重放为游戏过程视频与采样帧，并以评分量规引导的多模态评判对所得证据进行评分。

轨迹转换为游戏过程证据，最后依据隐藏评分量规对证据进行评分。这种结构使得开放式的游戏生成可以相互比较，而不必强制所有游戏共享相同的操控、关卡、目标或终止条件。

阶段一：任务打包。 每个任务打包三个对象：一段自然语言游戏规格、一个基于 Godot 的开发环境，以及一份隐藏评测评分量规。规格描述了预期的玩家体验，包括玩家幻想、核心循环、机制、目标、内容范围、视觉风格，以及预期的演示场景。环境提供了可编辑工作区、Godot 运行时与工具链、共享资源、文档、辅助工具以及提交契约。隐藏评分量规将同一意图分解为四个类别下的可观测要求：核心机制、内容深度、功能性视觉，以及美术与呈现。因此，智能体收到的是一份自然的游戏构建规格，而验证器保留着一个精确的评分目标，且该目标不会以清单形式暴露出来。

阶段二：智能体生成。 随后，智能体使用打包好的环境来构建一个 Godot 项目。它可以创建场景、编写脚本、配置输入动作、使用共享资源、运行项目、检视截图，并依据工具反馈进行迭代。所要求的提交包含两部分：一个完整的 Godot 游戏项目 G 和一组可重放的演示轨迹 Π 。项目是待评测的生成产物。轨迹则是评测产物：它们包含带时序的鼠标与键盘事件，可选地附带场景标识符，用以指定验证器应如何操作游戏以揭示相关的游戏过程行为。

阶段三：构建门控。 验证器首先检查所提交的项目与轨迹能否进入评测流水线。它解析项目结构与轨迹文件，在基准运行时启动该 Godot 项目，并检查游戏是否到达可运行状态。如果项目无法启动，或者没有任何提交的轨迹能够被解析，验证器就将 $BUILD = 0$ ，且最终得分为零。该门控确保只有在提交成为一个带有可重放交互证据的可执行产物之后，评分才会开始。

阶段四：重放。 对于每条有效轨迹，验证器都会启动一个全新的 Godot 实例，并在固定的 1280×720 视口中重放所记录的鼠标与键盘事件。重放是开放式交互与标准化证据之间的桥梁。如果没有提交的轨迹，验证器就必须先推断每个游戏应当如何被游玩，然后才能判断它是否可用，这会把验证变成一个自主探索问题，并使得分依赖于验证器的探索策略。通过重放演示，GameCraft-Bench 在固

定交互过程的同时，仍然能够在真实玩家输入下评测该产物。验证器为每次重放记录一段游戏过程视频，并以每秒两帧的速率采样帧。所得的视频与帧就是传递给评判器的证据。

阶段五：评分与聚合。 验证器以隐藏的任务评分量规对重放出的游戏过程证据进行评分。评分量规的类别植根于可游玩产物的原则 [11, 12]：遵循 MDA [11]，**核心机制**刻画规则与状态转移，**内容深度**刻画运行时范围与进程，而**美术与呈现**刻画作者性的感官层；遵循可玩性启发式 [12]，**功能性视觉**刻画游戏中的可读性与视觉反馈。每个演示都由一个多模态评判器使用采样帧、评测 prompt 与评分量规条目进行独立评分。随后验证器依据条目语义对演示级别的得分进行聚合：场景特定的要求采用最大值聚合，而持续性的要求采用均值聚合。聚合后的条目得分产生类别得分 M 、 D 、 V 与 A ，它们组合为

$$\text{Score} = \text{BUILD} \times (w_M M + w_D D + w_V V + w_A A),$$

其中这四个类别分别表示核心机制、内容深度、功能性视觉，以及美术与呈现，默认权重分别为 0.15, 0.35, 0.15, 0.35。我们优先考虑内容深度与美术与呈现，因为一个完整的游戏应当超越最低限度的机制与功能性原型。

3.3 它如何满足三项准则

基准要求	Godot	Unity	Unreal	GameMaker
开源	✓	×	×	×
轻量部署	✓	△	×	✓
命令行/无头友好	✓	△	△	△
原生 2D 工作流	✓	△	△	✓
基于文本的场景/文件	✓	△	△	×

表 2 用于自动化 2D 游戏生成基准测试的引擎适配性。△ 表示支持，但需要更繁重的部署、存在编辑器/工具链约束，或对轻量可复现评测的适配性较弱。

基于真实引擎：Godot（准则 I） **GameCraft-Bench** 使用 Godot 4 作为具体的引擎环境。其目标并非声称 Godot 是唯一可自动化的游戏引擎，而是选择一个能让大规模、可复现、基于执行的评测变得切实可行的引擎。Godot 是开源的、轻量的，提供原生的 2D 引擎栈，将场景存储为基于文本的项目文件，并支持命令行/无头执行以运行和导出项目。这些特性使验证器能够启动生成的游戏、重放轨迹、记录游戏过程证据并检视失败，而无需依赖 GUI 编辑器工作流。相比之下，Unity、Unreal 与 GameMaker 也以不同形式支持自动化，但它们引入了更繁重的安装、专有许可约束、以编辑器为中心的工作流、二进制项目格式，或者对轻量 2D 基准执行的适配性较弱。出于这一原因，**GameCraft-Bench** 使用 Godot 作为基准环境，并将多引擎评测留作未来工作。

完整游戏交付（准则 II） **GameCraft-Bench** 将生成的游戏产物 G 作为提交的单位。提交契约 C 要求智能体在目标工作区下放置一个完整的 Godot 项目，包括项目元数据、入口场景、脚本、复制好的资源、输入配置、UI 资源，以及启动游戏所需的运行时入口点。这防止智能体仅凭孤立的机制、零散的资源、不完整的场景，或需要额外人工组装的代码片段而获得分数。验证器以一道可启动性门控来强制执行该要求：如果项目无法在 Godot 运行时中启动，则 $\text{BUILD} = 0$ ，且最终得分为零。因此，**GameCraft-Bench** 评测的是智能体能否交付一个自包含的可游玩产物，而不仅仅是生成类游戏的组件。

交互式评测（准则 III） **GameCraft-Bench** 将评测建立在被观测到的玩家-游戏交互之上，而非静态检视。每次提交都包含可重放轨迹 Π ，验证器在 Godot 运行时中执行所提交的项目 G ，重放这些轨迹，并记录游戏过程观测 $O = \text{Replay}_{\mathcal{R}}(G, \Pi)$ 。随后将隐藏评分量规 ρ 应用于这一游戏过程证据，评

判所要求的机制、内容、视觉反馈与呈现是否在游玩过程中真正出现。这确保产物被作为交互系统来评测：源代码、项目结构、资源与截图对调试可能有用，但它们不能替代玩家输入下被观测到的行为。

3.4 任务集与标注质量

GameCraft-Bench 包含横跨 15 个游戏族的 140 个任务。这些游戏族的选择旨在覆盖各不相同的游戏构建需求，而不仅仅是表面的题材：平台跳跃与竞速游戏中的连续控制与碰撞、卡牌与策略游戏中的规则与状态管理、模拟经营与放置游戏中的进程与经济、开放世界与 Roguelike 游戏中的探索与空间布局，以及视觉小说、恐怖、音乐节奏与体育游戏中以叙事或呈现为主的交互。表 3 总结了游戏族级别的覆盖情况。

标注流程。 我们使用 Harbor [13] 作为任务编写框架。任务集由 12 名标注者标注，每位标注者都因其在多样题材上的丰富游玩经验而被选中。对于每个任务，标注者编写两份相互对齐的产物：一份公开的、产品风格的游戏规格，描述玩家幻想、核心循环、机制与美术方向；以及一份隐藏的评测评分量规，将同一游戏分解为可观测的、要求级别的标准。标注者确保规格是自然且开放式的——指定预期体验而不规定实现细节——同时使评分量规足够精确，以便能够从重放出的游戏过程证据中加以评分。给标注者的说明概述如下。

给标注者的说明

将每个 GameCraft-Bench 任务标注为三个文件：`task.toml`、`instruction.md` 和 `tests/rubric.json`。最终目标是评测一个智能体能否构建出一个完整、可交付的微型游戏，而不是一个原型、静态 UI 样稿或仅有标签的演示。

1. 元数据。 仅将 `task.toml` 用于元数据与运行时配置，例如任务名称、简短描述、关键词、验证器设置、智能体预算与环境设置。不要把游戏过程要求、演示规则或评分量规条目放入 `task.toml`。

2. 游戏规格。 将 `instruction.md` 作为关于智能体必须构建什么的事实来源。开头应当命名游戏、指定输出路径 `/workspace/game`，并将质量目标设定为一个完整、可交付的微型游戏。撰写一个 **Core Vision**（核心愿景）小节，描述玩家幻想与核心循环，以及一个 **What the Player Experiences**（玩家所体验到的）小节，描述面向玩家的弧线。规格应当描述可观测的玩家体验、有意义的决策、反馈、变化与结果，同时避免评分量规式的清单或实现级的指令。包含关于资源、项目布局、启动命令、演示、场景与轨迹格式的标准小节。不要要求音频相关要求，因为验证器评分的是视觉证据。

3. 评测评分量规。 使用 `tests/rubric.json` 将游戏规格转化为供验证器使用的具体评分量规条目。每份评分量规都应当包含 `score_formula`、`max_demos`、`max_demo_seconds`、`build_check`、`categories` 与 `requirements`。使用四个固定类别：核心机制、内容深度、功能性视觉，以及美术与呈现（Art & Presentation）。使用稳定的条目前缀：`M*`、`D*`、`V*` 与 `A*`。将评分量规条目的总数保持在 24 或更少。

4. 类别校准。 核心机制应当评测不可再约简的游戏过程循环是否能通过玩家动作、状态变化、规则、后果、目标与失败而运转。内容深度应当评测多样化的素材、进程、后期状态，或多个场景。功能性视觉应当评测游玩过程中的可读性与反馈，包括可读的状态、警告、目标，以及在 1280×720 下稳定的 UI。美术与呈现（Presentation & Art）应当评测游戏给人的感觉是经过作者打磨且可发布的，而非调试态的。

5. 评分校准与聚合。 满分应当要求可发布的纵向切片质量。当某一特性仅有标签、装饰性、静态、与游戏过程脱节、仅以名称或颜色表示、由默认控件主导，或主要由原始的程序化图形构成时，将弱证据的得分上限设为 0.5 或以下。对于应当在各演示间保持成立的属性使用 `mean`，例如可读性、视觉反馈、美术一致性、UI 重叠与呈现质量。仅当观测某一特性一次便足以证明其存在时，才使用 `max`，并保守地使用它，以避免对静态演示过度给分。

质量控制。 在规格与评分量规起草完成后，标注者进入验证阶段，通过编写一个简单的预言机（参考）解——一个在 Godot 中的最小可游玩草稿——来交叉核对任务的内部一致性。该预言机服务于三项验证功能：它验证规格在引擎中是否可实现、所要求的行为能否通过重放出的游戏过程加以展示，以及每个评分量规条目是否对应于一个可观测的状态，而非主观偏好。如果预言机无法到达预期的游戏过程状态，或者某个评分量规条目在重放中缺乏支撑证据，标注者就修订规格与评分量规并重新验证，直到任务变得内部一致。这种规格-评分量规-预言机的对齐确保了智能体面对一个定义良好的目标、评判器只评测可执行的行为，且最终得分反映了生成的产物是否实现了预期的玩家体验。

游戏族	任务数	游戏族	任务数	游戏族	任务数
平台跳跃	19	策略	17	模拟经营	16
开放世界	15	Roguelike	14	视觉小说	11
解谜	8	射击	7	模拟	6
卡牌	5	恐怖	5	音乐节奏	5
放置	4	竞速	4	体育	4

表 3 GameCraft-Bench 中的任务游戏族覆盖。计数不包括示例任务。

4 基准测评结果

实验设置。 我们在完整的 **GameCraft-Bench** 任务集上评测了七种前沿编码智能体配置：搭载 Opus-4.7 [14] high 与 MiMo-V2.5-Pro [15] 的 Claude Code [8], 搭载 GPT-5.5 [16] high 与 DeepSeek-V4-Pro [17] 的 Codex [7], 搭载 Kimi-K2.6 [6] 的 Kimi Code, 以及搭载 GLM-5.1 [18] 与 MiniMax-M2.7 [19] 的 Code Buddy。每种配置都在相同的基准接口与环境下运行全部 140 项任务。我们在附录 A 中提供了详细的评测设置。

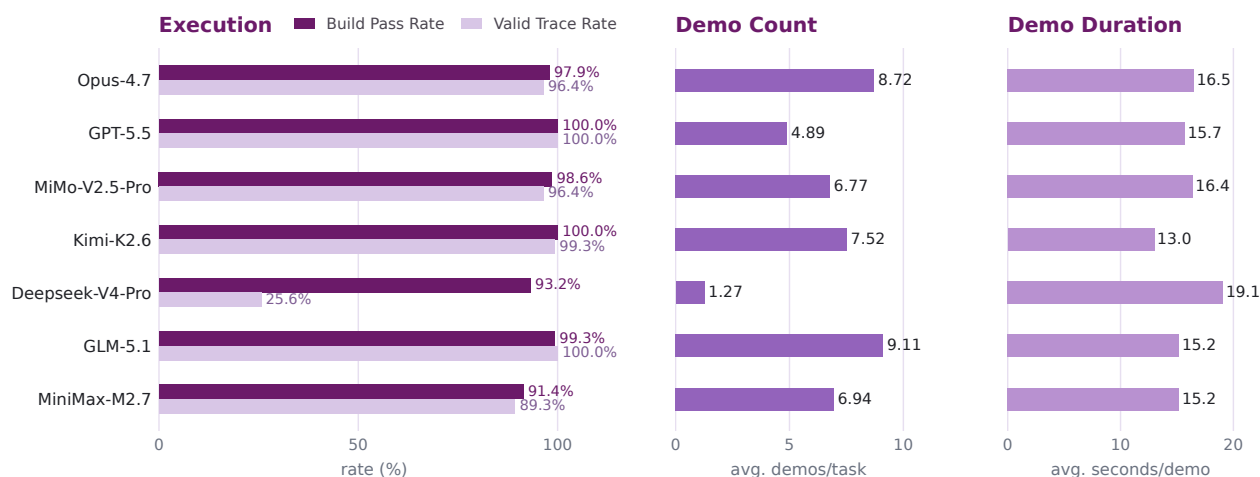


图 6 各智能体的执行与重放统计。构建通过衡量可启动性，有效轨迹衡量提交的演示中是否至少有一条可被重放，演示数量/时长则汇总了可用于评判的游戏过程证据的多少。

主要结果。 表 4 以百分点报告了基准层面的分数；详细的家族层面结果见附录 B。完成度最高的配置——Claude Code 下的 Opus-4.7 high——总体仅达到 41.46%。GPT-5.5 high 以 39.49% 紧随其后，而 Kimi-K2.6 达到 30.65%。MiMo-V2.5-Pro 降至 24.10%，DeepSeek-V4-Pro 则仅达到 2.15%。因此，即便是当前最优秀的编码智能体，距离可靠的端到端游戏生成仍相去甚远：它们往往能产出可运行的类游戏产物，但并不能始终如一地实现所要求的机制、内容、视觉状态与呈现。图 6 进一步将可启动性与可重放证据区分开来。大多数强力智能体都满足了基本的执行契约，但偏低的最终分数表明，产生一个可运行的项目以及可重放的交互轨迹，距离满足评分量规仍相去甚远。DeepSeek-V4-Pro 经常忽略或违反演示轨迹要求，从而在构建通过率与有效轨迹率之间产生了巨大差距。

分维度结果。 基准层面的各维度分数表明，机制始终强于内容与呈现。在较强的智能体中，Opus-4.7 high、GPT-5.5 high 与 Kimi-K2.6 在核心机制上分别获得 55.34%、54.36% 与 39.76%。同样这些智能体在内容深度上得分更低，分别为 39.48%、38.61% 与 28.07%，而 Kimi-K2.6 与 MiMo-V2.5-Pro 在美术与呈现上的得分更低。这一规律表明，智能体往往能创建出局部的交互循环，却难以将其扩展为具备充分状态推进、视觉可读性与精致呈现的完整游戏。

运行框架	模型	总体	机制	深度	视觉	美术
Claude Code	 Opus-4.7 high	41.46	55.34	39.48	42.78	36.86
	 MiMo-V2.5-Pro	24.10	32.33	22.59	27.45	20.65
Codex	 GPT-5.5 high	<u>39.49</u>	<u>54.36</u>	<u>38.61</u>	<u>41.84</u>	<u>32.94</u>
	 DeepSeek-V4-Pro	2.15	2.25	1.69	1.97	2.63
Kimi Code	 Kimi-K2.6	30.65	39.76	28.07	33.66	27.99
Code Buddy	 GLM-5.1	18.29	25.23	17.80	21.14	14.59
	 MiniMax-M2.7	10.95	14.27	9.92	14.92	8.85

表 4 基准层面的结果 (%)。机制、深度、视觉与美术对应四个评分量规维度。最佳与次佳分数分别以加粗与下划线标出。

发现 1.

智能体更常产出可识别的局部机制，但仍无法将其组装成完整、连贯的交互系统。

5 深入分析

5.1 论智能体的诊断模式

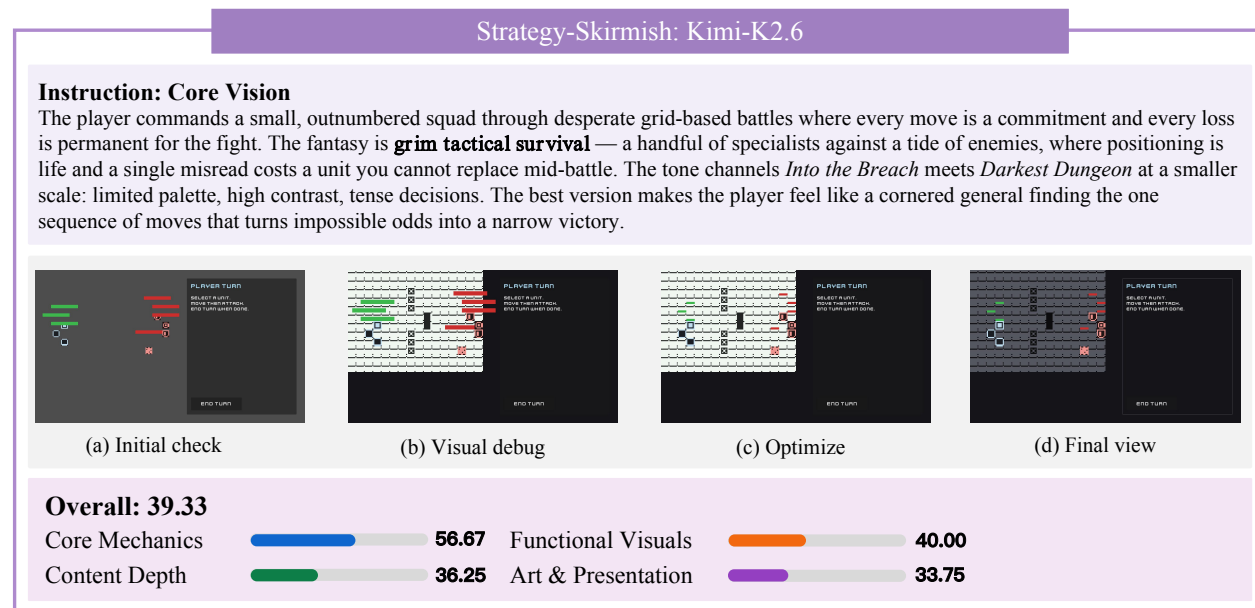


图 7 Kimi-K2.6 中感知引导的调试示例。对渲染后的游戏画面的反复检查使智能体能够识别并纠正面向玩家的失败。

成功信号：智能体会使用渲染反馈吗？ 游戏生成仅凭源代码和终端日志很难调试。许多失败只有在渲染项目之后才会显现：错误的摄像机取景、不可读的 UI、缺失的视觉反馈、损坏的关卡布局，或是未能展现预期机制的演示状态。在没有视觉反馈的情况下，智能体可能会继续编辑代码，却对面向玩家的游戏状态是否真正连贯一无所知。视觉交互改变了这一循环，它把渲染后的游戏画面转

化为一种调试信号：智能体可以检查屏幕，定位预期游戏状态与观察到的游戏状态之间的不匹配，并据此修订项目。Kimi-K2.6 提供了这一行为的一个具体例子。在 140 个 Kimi 任务中，我们统计到 2,998 次渲染屏幕检查，平均每个任务 21.41 次，中位数为 19；只有 4 个任务没有使用截图。同样的模式也出现在 Opus-4.7 中，它调用该辅助功能 1,952 次（每个任务 13.94 次），而 GPT-5.5 的使用则要节省得多，仅有 268 次调用（每个任务 1.91 次）。图 7 展示了来自 *Strategy-Skirmish*（策略—遭遇战）的一个代表性案例，其中 Kimi 反复检查渲染帧，调试错位的单位放置和缺失的选择高亮，并调整了网格和回合指示器的布局。这表明，视觉交互把游戏生成从一次性的代码合成转变为感知引导的迭代。

发现 2.

渲染交互帮助智能体调试那些在源代码中无法观察到的失败。

失败信号：更多工具有帮助吗？ 编写更多代码或运行更多命令不足以产生一个可玩的游戏。MiMo-V2.5-Pro 表现出一种典型的先写后跑、重度依赖 *bash* 的模式：智能体在进行任何验证之前，先快速搭建出一整套完整的文件集，包括 *project.godot*、GDScript 源文件和场景文件。随后它进入一个由 *shell* 命令主导的、漫长的执行驱动调试阶段，这些命令在全部 140 个任务的所有工具调用中占 56.3%，而代码读取和编辑操作（Read + Edit）仅占 16.5%。关键的是，工具使用总量与任务质量是脱钩的，如图 8 所示：一个典型任务涉及的工具调用中位数为 128 次，但在 *bash* 调试循环中投入更多努力并不会转化为更高的得分。瓶颈是结构性的，而非原始努力量：在五个零分任务中，全部都产出了有效的构建，但都没有提交演示轨迹，导致每一个量规维度都未被评分。先写策略前置了代码生成，但常常错过了闭合评估循环的演示轨迹交付，揭示出一个与原始编码能力正交的任务完成差距。

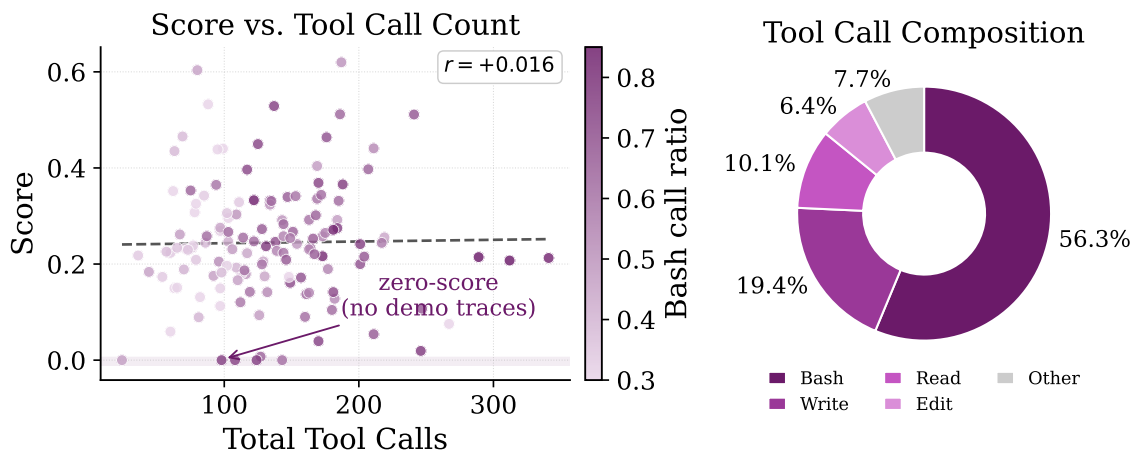


图 8 MiMo-V2.5-Pro 在 140 个任务中的工具使用情况。左：得分与工具调用总数的关系，按每个任务的 *Bash* 调用占比着色。近乎为零的相关性 ($r = +0.016$) 表明工具使用量与任务质量是脱钩的。右：工具调用的总体构成；*shell* 执行以 56.3% 占据主导，而代码读取和编辑仅占 16.5%。

发现 3.

执行努力和工具使用量是最终可玩性的弱预测因子；成功取决于闭合完整的构建-回放-评估循环。

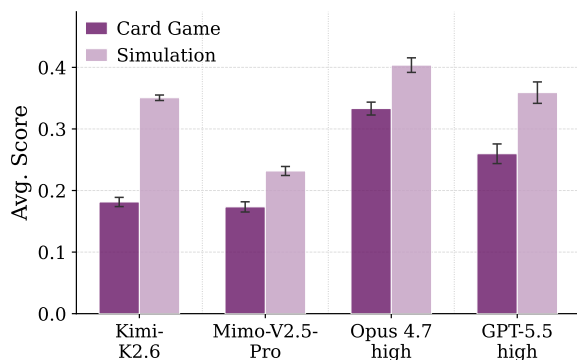


图 9 在固定游戏证据上的评判器稳定性。条形显示 10 次重复 GPT-5.5 评判器运行的均值得分，误差棒显示 ± 1 标准差。

5.2 论可玩性评判器的可靠性

稳定性：固定证据会得到一致的评分吗？ GameCraft-Bench 依赖于一个多模态评判器，因此我们首先测试相同的可玩证据在重复评估中是否得到稳定的评分。我们为来自两个游戏族（Simulation 和 Card Game）的任务固定其游戏证据（项目、轨迹、视频、帧和量规），并仅重新运行 GPT-5.5 评判器 $K = 10$ 次。如图 9 所示，均值得分在重复的 GPT-5.5 评估中保持一致，并且在所评估的各智能体配置和任务族之间误差棒都很小。定量地看，Kimi-K2.6 在 Card Game 上表现出 0.0037 的标准差，在 Simulation 上为 0.0038，而 Opus-4.7 high 则分别表现出 0.0050 和 0.0036 的标准差。这些波动远小于不同智能体和游戏族之间的性能差距，因此所观察到的排名对重复评判噪声是稳健的。残余噪声主要来自处于边界的视觉解读，例如某次卡牌交互或状态转换在视觉上是否足够清晰。

游戏族	人工	评判器	ΔM	ΔD	ΔV	ΔA	Δ 总体
卡牌	18.75	18.48	+2.33	+1.50	-6.10	-0.67	-0.27
放置	32.89	41.65	+6.25	+12.19	-3.12	+11.50	+8.76
竞速	17.42	19.81	-2.92	+3.44	-8.33	+8.20	+2.38
总体	22.69	26.02	+1.92	+5.38	-5.87	+5.80	+3.32

表 5 在 Kimi-K2.6 来自三个游戏族的提交上进行的初步人工校准。人工和评判器报告总体得分； Δ 各列报告评判器-人工之间以百分点计的差异。

校准：评判器与人工相比如何？ 我们还在来自三个游戏族的同一批回放游戏证据上进行了初步的人工校准。如表 5 所示，多模态评判器与人工评分大体一致，但总体上略偏宽松。主要的差异是维度特定的：人工标注者在内容深度（Content Depth）和美术与呈现（Art and Presentation）上更为严格，而评判器在功能性视觉（Functional Visuals）上更为严格。在游戏族层面，卡牌几乎完全吻合，而放置显示出最大的偏宽松差距，这表明未来的校准应聚焦于内容多样性和呈现方面的判断。由于这一子集规模很小，且并非为估计标注者间一致性而设计，我们将其视为一次校准检查，而非确定性的人工一致性研究。

发现 4.

可玩性评判器在重复评估中是稳定的，并在内容和呈现维度上表现出轻微的宽松性。

5.3 论游戏生成能力的可分解性

我们考察四个量规维度在 Kimi-K2.6 和 MiMo-V2.5-Pro 任务中是否同涨同落。如图 10 所示，对于 Kimi-K2.6，核心机制（Core Mechanics）与内容深度（Content Depth）（ $r = 0.61$ ）以及功能性视觉（Functional Visuals）（ $r = 0.53$ ）呈中等相关性，这表明更强的交互循环往往会暴露出更多的游戏状态和反馈。然而，美术与呈现（Art and Presentation）与其他维度的耦合要弱得多，尤其是与功能性视觉（ $r = 0.11$ ）。MiMo-V2.5-Pro 表现出类似但更具全局耦合性的模式，其美术与呈现与机制、内容深度和功能性视觉的相关性分别为 $r = 0.56$ 、 $r = 0.39$ 和 $r = 0.26$ 。这支持了主结果中的观察：一个机制上可识别的游戏并不会自动成为一个完整、视觉连贯或精致的游戏成品。



图 10 Kimi-K2.6 和 MiMo-V2.5-Pro 各量规维度之间的相关性。机制、深度和视觉呈中等耦合，而美术与呈现与其他维度的耦合更弱。

发现 5.

游戏生成能力是部分可分解的：机制、内容、视觉反馈和呈现并未完全耦合。

6 相关工作

编码智能体与软件工程评测。 编码智能体已经从诸如 CodeBERT [20] 和 StarCoder [21] 之类的代码补全模型，演进为能够进行仓库导航、多文件编辑和迭代调试的自主软件工程系统。近期的框架，包括 SWE-agent [5]、OpenHands [22]、ChatDev [23] 和 MetaGPT [24]，越来越多地在贴近现实的软件开发任务上对智能体进行评测。然而，现有的软件工程基准主要通过代码补丁、单元测试或问题修复结果来评估正确性，其假设是可执行行为能够被静态产物充分刻画。端到端的游戏生成对这一假设提出了挑战，因为成功与否取决于交互式运行时行为的质量，而不仅仅取决于源代码本身。

GUI 智能体与交互式评测。 GUI 智能体的近期进展使得模型能够通过视觉感知和原生界面动作，直接与桌面环境和 Web 环境进行交互。诸如 Mind2Web [25]、WebArena [26] 和 OSWorld [27] 之类的基准，强调在交互式设定下的端到端任务完成，而包括 CogAgent [28] 和 OSAtlas [29] 在内的系统，则展示出愈发出色的计算机操作行为。这些进展凸显了通过交互而非仅凭静态输出来评测智能体的重要性。然而，现有的 GUI 基准聚焦于完成预定义的界面任务，而非合成诸如游戏之类的可执行软件产物。

游戏生成基准。 近期工作已开始在游戏生成任务上对编码智能体进行评测，但现有基准覆盖了评测约定的不同部分。OpenGame-Bench [2] 要求智能体根据开放式提示生成完整的游戏，但它面向的是 Web 游戏，并且主要依赖静态或页面级的判定，而非玩法交互。GameDevBench [9] 将评测带

入了 Godot, 但研究的是在已有项目内由教程衍生的局部编辑, 而非完整游戏的端到端构建。与我们的设定最为接近的是 WebGameBench [10], 它通过真实的浏览器交互来评测所交付的浏览器原生游戏; 然而, 它仍处于引擎原生游戏开发之外, 并且依赖评测端的探索来发现玩法行为。关于 GameGen-Verifier [30] 的同期工作通过状态注入改进了运行时验证, 但并不评测从规格说明出发的开放式游戏构建。相比之下, **GameCraft-Bench** 同时要求智能体构建完整的 Godot 项目并提供可重放演示, 从而实现以交互为基础的评测, 而不会把验证本身变成一个自主的游戏探索问题。

7 结论

我们提出了一个面向端到端游戏生成的、以交互为基础的评测框架, 并将其实例化为 **GameCraft-Bench**——一个基于 Godot、面向编码智能体的基准。其核心论点是: 游戏生成不能像普通代码合成那样进行评测; 一个基准必须保留引擎环境、要求生成完整产物, 并通过交互来评判该产物。我们的结果表明, 当前的前沿智能体在这一契约之下仍远未达到可靠的水平。它们往往能产出可辨识的机制或可运行原型, 但在组装出具有充足内容、功能性反馈、呈现质量以及完整演示轨迹的连贯的游戏方面仍然力不从心。更广泛地说, 我们的发现表明, 对可执行的创意软件系统进行评测, 需要以交互为基础的评测, 而不能仅依赖静态代码检查、构建成功或视觉合理性。

致谢

本工作得到以下资助: 来自 Shenzhen Medical Academy of Research and Translation (SMART) 的 Major Frontier Exploration Program (Grant No. C10120250085)、Shenzhen Medical Research Fund (B2503005)、NSFC grant 72495131、1+1+1 CUHK-CUHK(SZ)-GDSTC Joint Collaboration Fund、Guangdong Provincial Key Laboratory of Mathematical Foundations for Artificial Intelligence (2023B1212010001), 以及 International Science and Technology Cooperation Center, Ministry of Science and Technology of China (under grant 2024YFE0203000)。

局限性

GameCraft-Bench 存在若干范围上的局限性。它聚焦于在 Godot 中进行的 2D 游戏生成, 这使得该基准在无界面 (headless) 评测中轻量且可复现, 但未涵盖诸如 Unity 与 Unreal 等其他主流引擎。它也不评测 3D 游戏、多人系统、大规模物理或长周期的生产工作流程。此外, 当前的验证器仅对视觉化的游戏过程证据进行评分; 节奏、恐怖或体育类游戏中依赖音频的方面, 是通过视觉行为来呈现的, 而非通过直接的音频评测。

GameCraft-Bench 也存在评测上的局限性。它依赖于一个多模态评判器来为回放的游戏过程证据评分, 因此结果仍可能受到评判模型偏差、API 漂移或视觉理解局限性的影响。最后, **GameCraft-Bench** 并不衡量一个生成的游戏在主观上是否有趣。相反, 它衡量的是智能体是否遵循游戏规格说明, 并在一个可执行产物中实现所要求的机制、内容、视觉状态与呈现。

参考文献

- [1] Yuxuan Wan, Runxin Yang, Shuqing Li, and Michael R. Lyu. 90% Faster, 100% Code-Free: MLLM-Driven Zero-Code 3D Game Development. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE 2026), Ideas, Visions and Reflections Track*, 2026.
- [2] Yilei Jiang, Jinyuan Hu, Qianyin Xiao, Yaozhi Zheng, Ruize Ma, Kaituo Feng, Jiaming Han, Tianshuo Peng, Kaixuan Fan, Manyuan Zhang, et al. Opengame: Open agentic coding for games. *arXiv preprint arXiv:2604.18394*, 2026.

- [3] Hongnan Ma, Han Wang, Shenglin Wang, Tieyue Yin, Yiwei Shi, Yucong Huang, Yingtian Zou, Muning Wen, and Mengyue Yang. CreativeGame: Toward Mechanic-Aware Creative Game Generation, 2026.
- [4] Lei Yin, Wentao Cheng, Zhida Qin, Tianyu Huang, Yidong Li, and Gangyi Ding. AutoUE: Automated Generation of 3D Games in Unreal Engine via Multi-Agent Systems. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2026), Findings, 2026.
- [5] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. Advances in Neural Information Processing Systems, 37:50528–50652, 2024.
- [6] Kimi Team, Tongtong Bai, Yifan Bai, et al. Kimi K2.5: Visual agentic intelligence, 2026.
- [7] Michael Bolin. Unrolling the codex agent loop. OpenAI Engineering Blog, January 2026.
- [8] Anthropic. Claude code overview. Official Documentation, 2026.
- [9] Wayne Chi, Yixiong Fang, Arnav Yayavaram, Siddharth Yayavaram, Seth Karten, Qiuhong Anna Wei, Runkun Chen, Alexander Wang, Valerie Chen, Ameet Talwalkar, et al. Gamedevbench: Evaluating agentic capabilities through game development. arXiv preprint arXiv:2602.11103, 2026.
- [10] Wenyu Zhang, Guoliang You, Haotian Zhao, Tianshu Zhu, Haoran Wang, Xiaoxuan Tang, Mingyang Dai, Jingnan Gu, Daxiang Dong, Jianmin Wu, et al. Webgamebench: Requirement-to-application evaluation for coding agents via browser-native games. arXiv preprint arXiv:2605.17637, 2026.
- [11] Robin Hunicke, Marc LeBlanc, Robert Zubek, et al. Mda: A formal approach to game design and game research. In Proceedings of the AAAI Workshop on Challenges in Game AI, volume 4, page 1722. San Jose, CA, 2004.
- [12] Heather Desurvire, Martin Caplan, and Jozsef A Toth. Using heuristics to evaluate the playability of games. In CHI’04 extended abstracts on Human factors in computing systems, pages 1509–1512, 2004.
- [13] Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, January 2026.
- [14] Anthropic. Claude Opus 4.7. Anthropic Blog Post, 2026.
- [15] Xiaomi MiMo Team. Mimo-v2.5-pro. <https://huggingface.co/collections/XiaomiMiMo/mimo-v25>, 2026.
- [16] OpenAI. GPT-5.5 system card. OpenAI Research, 2026.
- [17] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- [18] GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia’ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu,

- Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-5: from vibe coding to agentic engineering, 2026.
- [19] MiniMax. MiniMax-M2.7. Hugging Face Model Repository, 2026.
- [20] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. Codebert: A pre-trained model for programming and natural languages. In Findings of the association for computational linguistics: EMNLP 2020, pages 1536–1547, 2020.
- [21] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! arXiv preprint arXiv:2305.06161, 2023.
- [22] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In International Conference on Learning Representations, volume 2025, pages 65882–65919, 2025.
- [23] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers), pages 15174–15186, 2024.
- [24] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In International Conference on Learning Representations, volume 2024, pages 23247–23275, 2024.
- [25] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. Advances in Neural Information Processing Systems, 36:28091–28114, 2023.
- [26] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In International Conference on Learning Representations, volume 2024, pages 15585–15606, 2024.
- [27] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. Advances in Neural Information Processing Systems, 37:52040–52094, 2024.
- [28] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 14281–14290, 2024.
- [29] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: Foundation action model for generalist gui agents. In International Conference on Learning Representations, volume 2025, pages 5090–5108, 2025.
- [30] Chaobo Jia, Ruipeng Wan, Ting Sun, Weihao Tan, Borui Wan, Yuxuan Tong, Guangming Sheng, and Hong Xu. Gamegen-verifier: Parallel keypoint-based verification for llm-generated games via runtime state injection. arXiv preprint arXiv:2605.07442, 2026.

A 评测细节

运行环境。 **GameCraft-Bench** 基于 Harbor [13] 实现，并在本地子进程沙箱中运行每一次试验。参考配置使用 Ubuntu 22.04、Python 3.12 以及 Godot 4.6.2 stable。验证器依赖 Xvfb 实现无头显示，依赖 xdotool 进行合成输入，并依赖 ffmpeg 进行屏幕录制与帧提取。每个任务的智能体超时为 7200 秒，验证器超时为 1800 秒。除任务超时之外，我们不施加显式的工具调用次数上限。智能体获得一个以 /workspace/ 为根的固定工作区，并且必须将生成的 Godot 项目放置在 /workspace/game。

提交格式。 每次提交由一个 Godot 项目以及一到十个命名为 *.json 的演示轨迹文件组成，这些文件位于 /workspace/game/demo_outputs/ 下。仅按文件名排序的前十条轨迹会被评测。一条轨迹包含一个可选的场景设定标识符、以帧为单位的总重放时长，以及在固定 1280×720 视口中按时间排序的鼠标或键盘事件列表。演示可以确定性地初始化某个所请求的状态，例如一场战斗、升级界面或游戏后期的设置。轨迹模式如下：

演示轨迹模式 (Demo Trace Schema)

```
{
  "scenario": "title_flow",
  "duration_frames": 360,
  "events": [
    {"frame": 30, "type": "mouse_click",
     "button": "left", "x": 300, "y": 360},
    {"frame": 90, "type": "key_press", "keycode": "1"},
    {"frame": 180, "type": "key_press", "keycode": "SPACE"},
    {"frame": 300, "type": "wait"}
  ]
}
```

重放与评判。 对于每条有效轨迹，验证器启动一个全新的 Godot 实例，以每秒 30 帧的速率重放输入，通过 ffmpeg 录制 1280×720 的显示画面，并存储一个压缩的 854×480 版本。验证器每 0.5 秒采样一帧，对应每秒两帧。每个评分量规都可以限制演示的数量以及采样的时间窗口；默认基准配置最多使用十个演示，并对每个演示最多采样一个确定性的 20 秒窗口。我们使用 GPT-5.5 作为多模态评判器。评判器提示词模板如下所示：

评判器提示词模板

你是一名严格但公正的电子游戏评估者。给定一段简短的 Godot 2D 游戏游玩录像，判断所列出的每一条要求被清晰展示的程度。在 0.0 到 1.0 的尺度上为每一条要求打分，其中 0.0 = 完全没有展示（或与之相矛盾），0.5 = 部分展示／含糊不清，1.0 = 由录像中可见的内容清晰且无歧义地展示。仅以严格的 JSON 回复，不要散文，不要 markdown，不要代码围栏。

依据下列每一条要求评估该录像。

要求：

{requirements}

完全按照以下形状返回 JSON（不要额外的键，不要 markdown）：

```
{
  "scores": {"<requirement_id>": <0..1>},
  "rationales": {"<requirement_id>": "<one short sentence>"}
}
```

B 完整游戏族结果

模型	指标 (↑)	总体	平台跳跃	策略	模拟经营	开放世界	Roguelike	视觉小说	解谜	射击	模拟	卡牌	恐怖	音乐节奏	放置	竞速	体育
Claude Code																	
🌟 Opus-4.7 high	M	55.34	45.77	54.91	48.31	47.88	64.87	56.33	51.43	55.00	61.17	54.67	72.67	62.00	76.67	55.83	66.25
	D	39.48	35.50	46.78	39.42	34.43	38.79	42.22	26.43	40.57	37.33	26.25	49.00	40.00	61.25	45.31	40.62
	V	42.78	46.28	62.60	45.11	29.57	46.50	50.99	43.36	38.20	44.13	18.48	50.50	24.70	23.12	23.15	39.38
	A	36.86	29.55	30.65	24.69	48.94	28.05	35.21	35.44	34.58	26.93	38.92	61.92	55.90	58.82	50.15	51.95
	总体	41.46	36.57	44.73	36.45	40.80	40.10	43.60	35.87	40.28	38.29	33.78	57.30	46.57	56.99	45.26	48.24
👤 MiMo-V2.5-Pro	M	32.33	32.58	28.81	41.34	17.22	34.13	27.64	33.12	40.29	37.17	26.67	45.67	31.00	50.83	23.75	32.92
	D	22.59	17.50	20.11	36.31	17.08	21.27	21.02	21.09	19.00	28.17	12.25	26.00	13.75	41.25	16.56	38.75
	V	27.45	34.41	29.41	39.42	11.81	35.85	29.63	32.02	30.10	30.59	4.95	28.00	16.34	11.88	9.51	19.38
	A	20.65	18.46	14.12	19.53	22.96	18.19	15.37	21.21	25.02	20.53	14.59	41.14	16.69	42.94	24.57	29.29
	总体	24.10	22.64	20.72	31.66	18.37	24.31	21.36	24.58	25.96	27.21	14.14	34.55	17.76	38.87	19.39	31.66
Codex																	
🌀 GPT-5.5 high	M	54.36	50.51	50.96	42.64	54.48	50.99	56.00	67.97	64.00	53.83	50.67	59.67	57.33	90.83	57.50	52.08
	D	38.61	34.66	40.69	36.85	39.23	32.60	45.61	37.97	33.71	37.50	23.25	36.50	40.75	72.19	35.00	55.62
	V	41.84	45.49	56.61	41.01	27.29	43.42	50.46	53.34	44.89	40.10	17.73	44.00	32.84	28.12	25.23	33.75
	A	32.94	30.65	26.94	17.45	40.51	25.65	28.50	43.75	37.62	33.11	20.29	59.20	42.45	62.29	35.69	49.61
	总体	39.49	37.26	39.81	31.55	40.17	34.55	42.15	46.80	41.30	38.81	25.50	49.05	42.65	64.91	37.15	49.71
👤 DeepSeek-V4-Pro	M	2.25	0.39	1.35	0.00	0.92	1.71	0.45	0.00	0.00	6.50	0.00	7.33	0.00	33.33	0.00	8.33
	D	1.69	1.00	0.35	0.00	0.00	1.00	0.82	0.00	0.00	6.33	0.00	5.00	0.00	13.44	0.00	18.12
	V	1.97	2.67	0.88	0.00	1.11	3.48	2.56	0.00	0.00	5.62	0.00	2.00	0.00	6.88	0.00	11.25
	A	2.63	2.73	0.59	0.00	0.28	2.75	0.91	0.00	0.00	4.93	7.00	10.40	4.83	17.92	0.00	10.38
	总体	2.15	1.77	0.66	0.00	0.40	2.09	1.06	0.00	0.00	5.76	2.45	6.79	1.69	17.01	0.00	12.91
Kimi Code																	
📄 Kimi-K2.6	M	39.76	34.32	47.65	38.50	26.99	52.31	45.73	30.47	43.00	47.50	29.00	34.67	41.00	64.58	20.00	39.17
	D	28.07	22.71	32.22	32.04	23.10	33.22	40.09	18.12	30.43	40.83	11.50	17.50	14.50	38.75	8.12	44.38
	V	33.66	39.27	44.17	39.30	19.05	43.30	46.49	35.51	29.65	38.42	10.57	19.00	22.91	15.62	8.33	25.62
	A	27.99	28.43	21.23	22.60	24.62	26.27	27.75	26.23	34.61	25.60	24.33	42.19	31.84	45.87	36.32	46.74
	总体	30.65	28.94	32.48	30.79	23.61	35.17	37.85	25.42	33.66	36.14	18.48	28.94	25.80	41.65	19.81	41.61
Code Buddy																	
📄 GLM-5.1	M	25.23	22.37	19.12	27.25	22.53	30.09	21.45	35.62	24.29	42.67	18.67	25.00	15.33	40.00	33.33	12.92
	D	17.80	11.95	18.97	22.21	16.90	19.06	16.27	20.94	16.29	30.00	14.25	10.50	14.25	24.06	8.12	25.62
	V	21.14	22.35	25.85	26.40	11.75	25.12	24.33	29.49	20.37	34.58	7.91	13.00	12.13	11.25	7.86	12.50
	A	14.59	11.99	9.19	9.01	17.26	13.35	12.26	18.91	15.26	20.17	14.01	36.68	11.76	26.57	14.05	19.97
	总体	18.29	15.09	16.60	18.98	17.10	19.62	16.88	23.71	17.74	29.15	13.88	22.21	13.22	25.41	13.94	19.77
👤 MiniMax-M2.7	M	14.27	9.96	13.24	20.69	2.98	21.00	12.91	4.53	21.14	30.50	16.33	17.67	9.00	22.92	14.58	10.00
	D	9.92	5.00	10.51	14.56	4.98	11.72	12.18	5.78	11.86	20.17	3.00	12.25	11.00	11.88	9.69	10.31
	V	14.92	15.87	17.84	25.63	6.67	20.48	16.89	16.51	17.15	29.12	1.20	2.00	4.67	3.75	2.60	2.50
	A	8.85	6.37	4.13	7.48	9.34	6.84	5.77	3.04	13.29	15.25	10.31	25.19	14.24	21.79	12.00	8.94
	总体	10.95	7.85	9.79	14.66	6.46	12.72	10.82	6.24	14.55	21.34	7.29	16.05	10.88	15.78	10.17	8.61

表 6 游戏族层级与基准层级的结果 (%)。对每个模型，各行报告核心机制 (M)、内容深度 (D)、功能性视觉 (V)、美术与呈现 (A) 以及总体分数。

C 案例研究

Strategy-Skirmish 的 rubric.json 示例

```
"score_formula": "BUILD * (0.15*((M1+M2+M3+M4)/4) + 0.35*((D1+D2+D3+D4)/4) + 0.15*((V1+V2+V3+V4)/4) + 0.35*((A1+A2+A3+A4)/4))",
"max_demos": 10, "max_demo_seconds": 20,
"build_check": {
  "id": "BUILD", "cmd": "godot --headless --path /workspace/game --quit-after 5" },
"categories": [
  {"name": "Core Mechanics", "items": ["M1", "M2", "M3", "M4"]},
  {"name": "Content Depth", "items": ["D1", "D2", "D3", "D4"]},
  {"name": "Functional Visuals", "items": ["V1", "V2", "V3", "V4"]},
  {"name": "Art & Presentation", "items": ["A1", "A2", "A3", "A4"]} ],
"requirements": [
  {
    "id": "M1", "agg": "max",
    "description": "The irreducible tactical loop is playable: the player can select a friendly unit, see a limited legal movement range highlighted on the grid, choose a legal destination, and watch the unit move to the chosen cell. Illegal clicks outside the range are rejected without corrupting state. Selecting a different unit or clicking the same unit again cancels the current selection. Score 0 if selecting and moving a unit is missing, scripted-only, or not controlled by player input. Score 1 requires the mechanic to be presented in a visually authored context with real sprites or illustrated assets. Score 0.5 at most if the mechanic works but is represented entirely by programmatic shapes, solid-color fills, or default Godot widgets."
  },
  ...
  {
    "id": "D1", "agg": "max",
    "description": "The player's squad has meaningful depth: at least three controllable unit types or named characters are present, visually identifiable, and functionally different through HP, movement range, attack range, damage, or special ability. Score 1 only if the differences change tactical choices during battle. Score 0 if there is only one player unit type or if all units share identical stats and behavior. Score 0.5 at most if only labels or decorative differences distinguish units with no observable gameplay impact. Score 1 requires content variants to be visually distinct with real authored art (different sprites, illustrated elements). Score 0.5 at most if variants exist but are represented by programmatic shapes, color swaps, or text labels without real asset differences."
  },
  ...
  {
    "id": "V1", "agg": "mean",
    "description": "Battle readability is strong throughout gameplay: grid cells, walkable terrain, blocked terrain, friendly units, enemy units, selected unit, legal move cells, and attackable targets are all visually distinct at 1280x720. Score 1 only if a player can understand legal actions without guessing. Score 0 if key states blend together, overlays are missing, or placeholder shapes make sides and zones ambiguous. Score 1 requires the information to be presented through themed, styled UI with real assets. Score 0.5 at most if readability relies on plain text, unstyled labels, or raw geometric shapes."
  },
  ...
  {
    "id": "A1", "agg": "mean",
    "description": "The game has a coherent dark-fantasy art direction: terrain, units, UI, effects, and backgrounds share one consistent visual language with a deliberate limited palette and high contrast. Score 1 only if the presentation looks like an authored vertical slice rather than a prototype. Score 0 if visible gameplay relies on raw ColorRect shapes, default Godot widgets, mismatched asset packs, or bright off-brief placeholder colors."
  },
  ...
]
```


Strategy-Skirmish 的 instruction.md 示例

Strategy: Skirmish

Build a **dark-fantasy tactical skirmish** in Godot 4 at ``/workspace/game/``. This is not a prototype. It is a **complete, shippable micro-game** that could sit on an itch.io page or Steam as a polished vertical slice.

Core Vision

The player commands a small, outnumbered squad through desperate grid-based battles where every move is a commitment and every loss is permanent for the fight. The fantasy is **grim tactical survival** — a handful of specialists against a tide of enemies, where positioning is life and a single misread costs a unit you cannot replace mid-battle. The tone channels *Into the Breach* meets *Darkest Dungeon* at a smaller scale: limited palette, high contrast, tense decisions. The best version makes the player feel like a cornered general finding the one sequence of moves that turns impossible odds into a narrow victory.

What the Player Experiences

A moody title screen sets the dark-fantasy tone immediately. The player begins and receives a brief tactical briefing — the squad's objective, the threat ahead, the stakes — before the grid appears.

The battle is turn-based and deliberate. The player selects a unit, sees its limited movement range light up on the grid, and commits it to a position. Enemies are visible, aggressive, and numerous — the squad is always outnumbered. After the player spends their actions, an End Turn command hands control to the enemy, which advances with purpose: flanking, closing distance, attacking when in range. Then control returns and the cycle repeats.

Combat is lethal and readable. Attacks require proximity or a clear range indicator, reduce persistent HP, and kill. Dead units vanish from the board and stop blocking or threatening. The player's squad members are specialists — different movement ranges, attack patterns, HP pools, or abilities — so choosing who moves where and who attacks what is the core decision space.

The battlefield itself adds tactical texture: terrain obstacles funnel movement, hazards punish careless positioning, or objectives create time pressure beyond simple elimination. Multiple battle layouts keep the experience from feeling solved after one fight.

Victory comes from eliminating all enemies; defeat from losing the squad. Either outcome lands on a styled result screen showing what happened, and the player can retry or return to the title without relaunching. The entire arc — title, briefing, battle, result — flows as one continuous authored experience.

{Resource Instruction}

{Foramt Instruction}

instruction.md 中的资源说明

Assets

2D assets are mounted read-only at:

- ``/workspace/assets/library/`` — Kenney CC0 packs (sprites, tiles, UI, fonts).
- ``/workspace/assets/library-oga/`` — OpenGameArt entries; respect each subdir's ``LICENSE.txt``.

Browse the library and choose packs.

Copy what you need into your project's ``assets/`` folder.

Project layout

```
...  
/workspace/game/  
  project.godot  
  Main.tscn  
  demo_outputs/    ← your input traces (1-10 files)  
  scripts/  scenes/  assets/  
...
```

The build must launch cleanly with:

```
...  
godot --headless --path /workspace/game --quit-after 5  
...
```

A reference for Godot CLI flags is at ``/workspace/tools/godot_command_line.md``.
**Engine flags like ``--headless`` and ``--quit-after N`` must come BEFORE ``--`` — anything after ``--`` is forwarded to the project as user args and silently ignored by the engine. Correct shape:
``godot --headless --quit-after 5 --path . -- --scenario near_victory``.

A screenshot helper is available at ``/workspace/tools/screenshot.sh``.
Use it to actually see what your UI / battlefield / result screens look like.

```
...  
/workspace/tools/screenshot.sh --path /workspace/game \  
  -- --out /workspace/frame.png --frames 60  
...
```

To screenshot a specific scenario, append ``--scenario <id>`` after ``--``. The helper consumes only ``--out`` / ``--frames`` / ``--scene``; remaining args stay in ``OS.get_cmdline_user_args()`` for your game code to read. Example:

```
...  
/workspace/tools/screenshot.sh --path /workspace/game \  
  -- --out /workspace/battle_debug.png --frames 120 --scenario battle  
...
```

instruction.md 中的格式说明

Demos

Ship ****1-10 input-trace files**** under `~/workspace/game/demo_outputs/``, one per demo, each named ``.json``. The evaluator launches a fresh game per trace, replays your trace as synthetic mouse and keyboard input at 1280×720, and records the screen. Only the first 10 traces by filename are evaluated; recordings longer than 20 s are sampled from a random 20 s window.

Scenarios

Normal play should start from the title screen and demonstrate the task's core gameplay loop.
Demo playback must be deterministic. For demos that need a specific state (a specific level, combat state, upgrade screen, result state, or late-game setup), define named scenarios your game loads when launched with:

```
godot --path /workspace/game -- --scenario <id>
```

When `--scenario <id>` is present the game must skip menus, set up the named state deterministically (seed any RNG), and begin accepting input immediately.

Trace file format

```json

```
{
 "scenario": "title_flow",
 "duration_frames": 360,
 "events": [
 {"frame": 30, "type": "mouse_click", "button": "left", "x": 300, "y": 360},
 {"frame": 90, "type": "key_press", "keycode": "1"},
 {"frame": 180, "type": "key_press", "keycode": "SPACE"},
 {"frame": 300, "type": "wait"}
]
}
```

---

- ``scenario`` — optional; omit for a normal game launch from the title screen.
- ``duration_frames`` — total frames to record at 30 fps; cap at **\*\*600 (20 s)\*\***.
- ``events`` — time-ordered inputs. Coordinates are pixels in the 1280×720 viewport. Supported types:
  - ``mouse_click``: `{frame, type, button: "left"|"right", x, y}``
  - ``mouse_down`` / ``mouse_up``: `{frame, type, button: "left"|"right", x, y}`` — use these for drag interactions: emit ``mouse_down`` at the start point, one or more ``mouse_move`` events along the way, and ``mouse_up`` at the end. A ``mouse_click`` is a ``mouse_down`` + ``mouse_up`` at the same point in tight succession.
  - ``mouse_move``: `{frame, type, x, y}``
  - ``key_press`` / ``key_down`` / ``key_up``: `{frame, type, keycode}`` — keycodes: ``A`-`Z``, ``0`-`9``, ``ESCAPE``, ``ENTER``, ``SPACE``, ``TAB``, ``BACKSPACE``, ``DELETE``, ``SHIFT``, ``CTRL``, ``ALT``, ``UP``, ``DOWN``, ``LEFT``, ``RIGHT``.
  - ``wait``: `{frame, type}`` — anchor frame, no input.

Replay must be deterministic: same trace, fresh launch, same outcome every time.

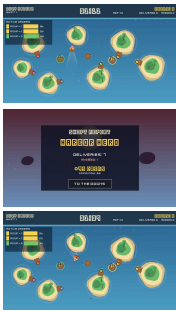
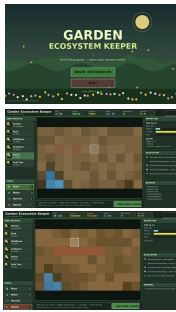
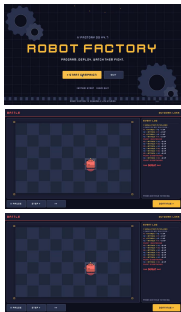
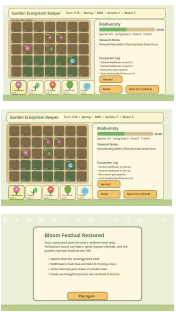
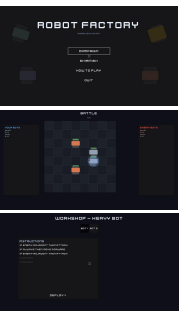
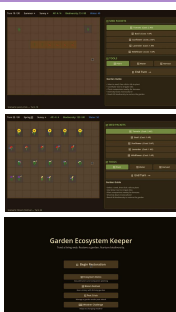
|                                                                                                          | Cozy Harbor Delivery                                                                                                                        | Garden Ecosystem Keeper                                                                                                                     | Space Station                                                                                                                                | Robot Factory                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <br>Opus-4.7<br>High    | <br>M: 0.74 D: 0.78<br>V: 0.85 A: 0.72<br>Overall: 0.763   | <br>M: 1.00 D: 0.82<br>V: 0.58 A: 0.51<br>Overall: 0.705   | <br>M: 0.44 D: 0.22<br>V: 0.51 A: 0.28<br>Overall: 0.316   | <br>M: 0.74 D: 0.38<br>V: 0.70 A: 0.33<br>Overall: 0.463   |
| <br>GPT-5.5<br>High     | <br>M: 0.60 D: 0.61<br>V: 0.70 A: 0.58<br>Overall: 0.612   | <br>M: 0.56 D: 0.57<br>V: 0.37 A: 0.46<br>Overall: 0.502   | <br>M: 0.44 D: 0.37<br>V: 0.44 A: 0.33<br>Overall: 0.377   | <br>M: 0.80 D: 0.47<br>V: 0.78 A: 0.41<br>Overall: 0.545   |
| <br>Kimi-<br>K2.6     | <br>M: 0.50 D: 0.45<br>V: 0.43 A: 0.12<br>Overall: 0.368 | <br>M: 0.60 D: 0.53<br>V: 0.46 A: 0.34<br>Overall: 0.463 | <br>M: 0.60 D: 0.50<br>V: 0.69 A: 0.40<br>Overall: 0.485 | <br>M: 0.47 D: 0.29<br>V: 0.43 A: 0.16<br>Overall: 0.27  |
| <br>MiMo-<br>V2.5-Pro | <br>M: 0.44 D: 0.44<br>V: 0.43 A: 0.23<br>Overall: 0.386 | <br>M: 0.68 D: 0.65<br>V: 0.41 A: 0.41<br>Overall: 0.536 | <br>M: 0.26 D: 0.17<br>V: 0.23 A: 0.12<br>Overall: 0.194 | <br>M: 0.27 D: 0.35<br>V: 0.15 A: 0.10<br>Overall: 0.217 |

图 11 案例研究：四个模型在四项代表性任务上的表现。每个单元格展示来自智能体所提交演示轨迹的三帧采样游玩画面，连同各类别分数（ $M$ ：核心机制， $D$ ：内容深度， $V$ ：功能性视觉， $A$ ：美术与呈现），以及在下方显示的总体分数。